

Lab02_Diego.QMD

Lab02_182745

1) Quais são as estatísticas suficientes para a determinação do percentual de vôos atrasados na chegada ($\text{ARRIVAL_DELAY} > 10$)?

1. **n_total**: O número total de voos válidos no lote. 2. **n_atrasos**: O número total de voos com atraso superior a 10 minutos ($\text{ARRIVAL_DELAY} > 10$) no lote.

Com essas estatísticas agregadas por lote, somam-se os totais ao final e calcula-se a taxa exata sem perdas: $\text{Perc} = \text{Soma}(\text{N_atrasos}) / \text{Soma}(\text{N})$

```
library(dplyr)
```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

filter, lag

The following objects are masked from 'package:base':

intersect, setdiff, setequal, union

```
library(readr)
```

Criando a Função getStats

2. Crie uma função chamada `getStats` que, para um conjunto de qualquer tamanho de dados provenientes de `flights.csv.zip`, execute as seguintes tarefas (usando apenas verbos do `dply`)

```
# a. Filtre o conjunto de dados de forma que contenha apenas observações das seguintes Cias.
# b. Remova observações que tenham valores faltantes em campos de interesse;
# c. Agrupe o conjunto de dados resultante de acordo com: dia, mês e cia. aérea;
# d. Para cada grupo em b., determine as estatísticas suficientes apontadas no item 1. e o
# e. A função deve receber apenas dois argumentos:
#     i. input: o conjunto de dados (referente ao lote em questão);
#     ii. pos: argumento de posicionamento de ponteiro dentro da base de dados. Apesar de e
```

```
getStats <- function(input, pos) {

  cias <- c("AA", "DL", "UA", "US")

  input %>%
    filter(
      AIRLINE %in% cias,
      !is.na(ARRIVAL_DELAY),
      !is.na(MONTH),
      !is.na(DAY)
    ) %>%
    mutate(foi_atrasado = ifelse(ARRIVAL_DELAY > 10, 1, 0)) %>%

    group_by(MONTH, DAY, AIRLINE) %>%

    summarise(
      n_total = n(),
      n_atrasos = sum(foi_atrasado),
      .groups="drop"
    )

}
```

3. Utilize alguma função `readr::read_***_chunked` para importar o arquivo `flights.csv.zip`

```
# a. Configure o tamanho do lote (chunk) para 100 mil registros;
# b. Configure a função de callback para instanciar DataFrames aplicando a função getStats
# c. Configure o argumento col_types de forma que ele leia, diretamente do arquivo, apenas
col_interesse <- cols_only(
  MONTH = col_integer(),
  DAY = col_integer(),
  AIRLINE = col_character(),
  ARRIVAL_DELAY = col_double()
)
```

```
stats_lote <- read_csv_chunked(
  file = "flights.csv",
  callback = DataFrameCallback$new(getStats),
  chunk_size = 100000,
  col_types = col_interesse
)
```

4. Crie uma função chamada `computeStats`

```
# 4. Crie uma função chamada computeStats que:
#   a. Combine as estatísticas suficientes para compor a métrica final de interesse (percentual de atraso)
#   b. Retorne as informações em um tibble contendo apenas as seguintes colunas:
#       i. Cia : sigla da companhia aérea;
#       ii. Data : data, no formato AAAA-MM-DD (dica: utilize o comando as.Date);
#       iii. Perc : percentual de atraso para aquela cia. aérea e data, apresentado como um número
computeStats <- function(bruto_stats) {
  bruto_stats %>%
    group_by(AIRLINE, MONTH, DAY) %>%
    summarise(
      total_voos = sum(n_total),
      total_atrasos = sum(n_atrasos),
      .groups = "drop"
    ) %>%
    mutate(
      Cia = AIRLINE,
      Data = as.Date(paste(2015, MONTH, DAY, sep = "-")),
      Perc = total_atrasos / total_voos
    ) %>%
    select(Cia, Data, Perc)
}

final_stats <- computeStats(stats_lote)
```

5. Produza um mapa de calor em formato de calendário para cada Cia. Aérea.

```
library(ggplot2)
library(ggcal)
pal <- scale_fill_gradient(low = "#4575b4", high = "#d73027")

baseCalendario <- function(stats, cia) {
  dados_cia <- stats %>% filter(Cia == cia)
```

```

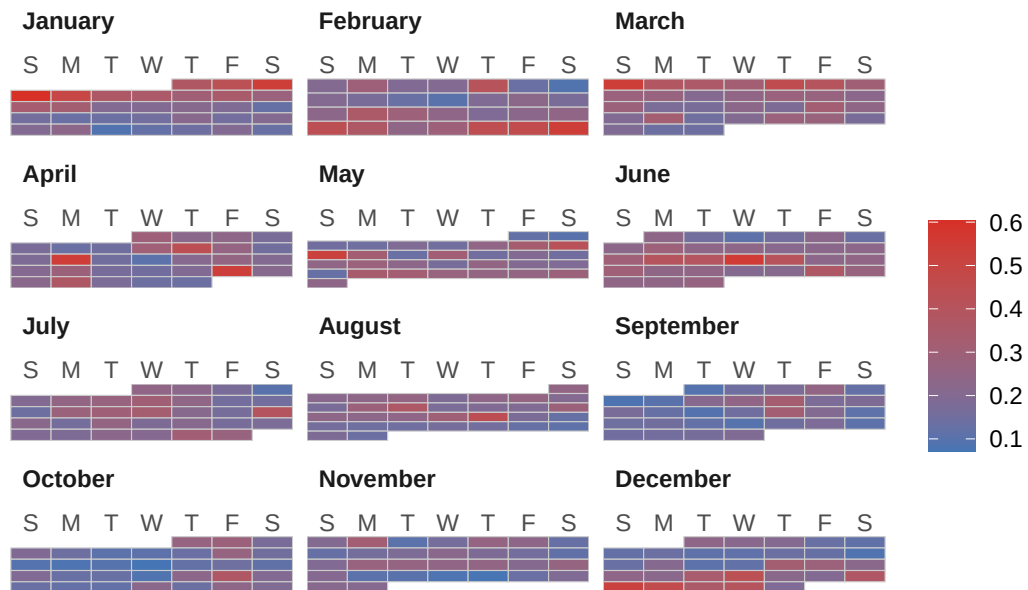
  ggcal(dados_cia$Data, dados_cia$Perc)
}

cAA <- baseCalendario(final_stats, "AA")
cDL <- baseCalendario(final_stats, "DL")
cUA <- baseCalendario(final_stats, "UA")
cUS <- baseCalendario(final_stats, "US")

cAA + pal + ggtitle("Percentual de Atraso na Chegada - American Airlines (AA)")

```

Percentual de Atraso na Chegada – American Airlines (AA)

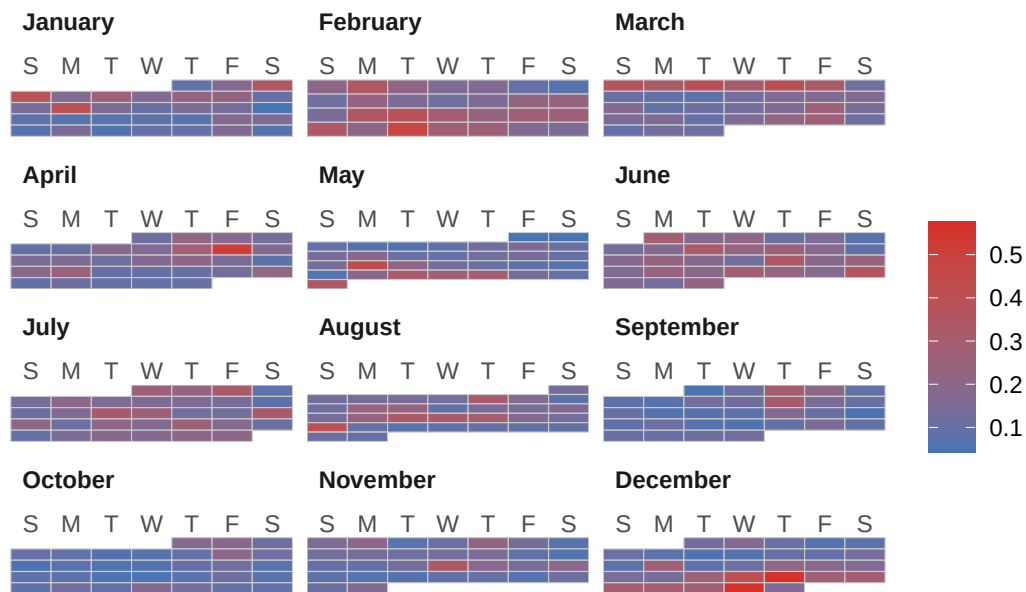


```

cDL + pal + ggtitle("Percentual de Atraso na Chegada - Delta Air Lines (DL)")

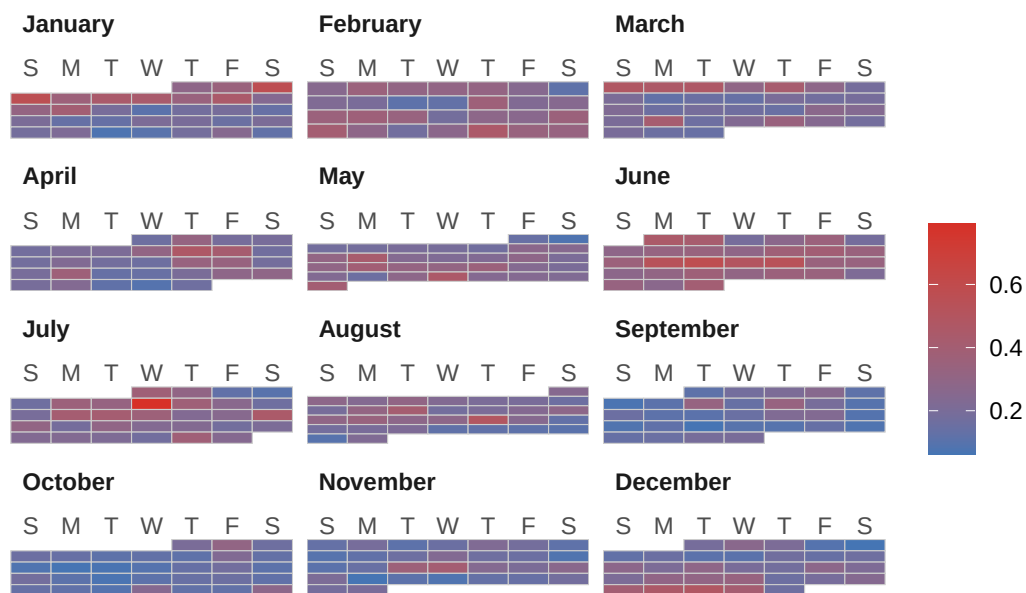
```

Percentual de Atraso na Chegada – Delta Air Lines (DL)



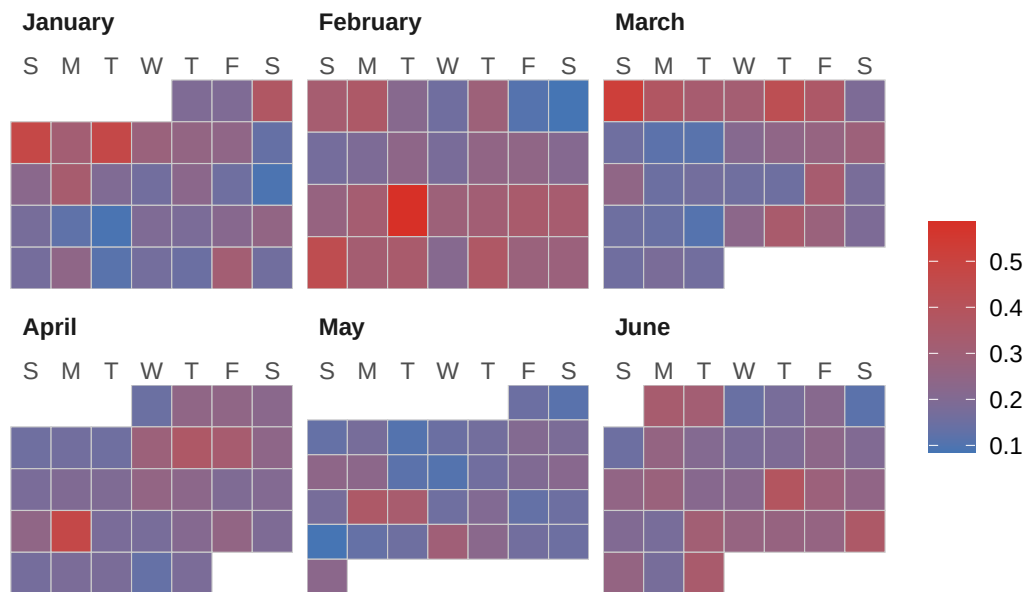
```
cUA + pal + ggtitle("Percentual de Atraso na Chegada - United Airlines (UA)")
```

Percentual de Atraso na Chegada – United Airlines (UA)



```
cUS + pal + ggtitle("Percentual de Atraso na Chegada - US Airways (US)")
```

Percentual de Atraso na Chegada – US Airways (US)



Hora de compilação

```
Sys.setenv(TZ = "America/Sao_Paulo")  
Sys.time()
```

```
[1] "2026-08-25 09:26:36 -03"
```